

KARP-RABIN fingerprints

How to quickly check files using fingerprints?

How to check for computer viruses?

Idea | Binary alphabet $\{0,1\}$

sequence $S = 0111010011100111101$ $n = |S|$

it can be seen as a very very large number $N_S \in [0 \dots 2^n - 1]$

sequence $S' = 01110 \dots$ $n = |S'|$

It costs $O(n)$ to check equality

$F(S) = N_S \% p$ for a random prime $p \in [2 \dots \tau]$ where $\tau \ll 2^n$
 $\uparrow$ we chooses it

Issue

$$F(S) \neq F(S') \Rightarrow S \neq S' \quad \checkmark$$

$$F(S) = F(S') \begin{cases} \rightarrow S = S' \quad \checkmark \\ \leftarrow S \neq S' \end{cases}$$

$S \neq S' \leftarrow$ 1-sided error

undevitable : what is the probability?

ERROR

$$\begin{array}{l} K_1 \rightsquigarrow N_S \\ K_2 \rightsquigarrow N_{S'} \end{array}$$

$$K_1 \neq K_2 \wedge (K_1 \% p = K_2 \% p)$$

Idea • choose τ sufficiently large (see later on)

• choose uniformly and randomly a prime in $[2.. \tau]$

Before starting the computation

p is a BAD prime if $K_1 \% p = K_2 \% p$

example: $K_1 = 37$

$K_2 = 40$

$p=3$ is BAD $p=5$ is ok

$$\Pr(\text{error}) = \Pr(k_1 \neq k_2 \wedge (k_1 \% p = k_2 \% p)) = \frac{\# \text{ BAD primes}}{\# \text{ primes in } [2..Z]}$$

$$= \frac{?}{Z / \ln Z} \leftarrow \text{density of primes} \leq \frac{n}{Z / \ln Z}$$

With high probability $= \frac{1}{n^c}$ $c = \text{constant} > 1$

to fix Z , it suffices to set $\frac{n}{Z / \ln Z} \leq \frac{1}{n^c}$

$$\Rightarrow Z \sim n^{c+1} \ln n$$

Fact Any number x with n bits has $< n$ distinct primes dividing x

Proof Factorization theorem

$$x = p_1^{e_1} p_2^{e_2} \dots p_r^{e_r}$$

$$2025 = 3^4 \times 5^2 \\ p_1^{e_1} \times p_2^{e_2}$$

$$p_i \geq 2 \Rightarrow e_1 + e_2 + \dots + e_r \leq n$$

as otherwise $x \geq 2^{e_1 + e_2 + \dots + e_r} > 2^n$ not possible

since $e_1 + e_2 + \dots + e_r \leq n \Rightarrow r = \# \text{ distinct primes dividing } x \leq n$

□

p is BAD if it divides both k_1 and $k_2 \Rightarrow$ at most n BAD primes

What if $k_1 \% p = k_2 \% p$ but p does not divide k_1, k_2 ?

We can apply this same argument to $(k_1 - k_2) = k$

37 and 40 are not multiple of 3

but $40 - 37$ is so.



APPLICATION : ROLLING HASHING

<https://github.com/lemire/rollinghashcpp>

TEXT STRING ABRACADABRA
 $m=4$
 BRAC
 $m=4$

$O(m)$ time

$$F(\text{BRAC}) = B \cdot 131^3 + R \cdot 131^2 + A \cdot 131 + C \cdot 131^0 \% p \quad \text{ASCII}$$

$$F(\text{RACA}) = R \cdot 131^3 + A \cdot 131^2 + C \cdot 131 + A \cdot 131^0 \% p$$

$$F(\text{RACA}) = (F(\text{BRAC}) - B \cdot 131^3) \cdot 131 + A \% p$$

↑

$O(1)$ time ...

STRING MATCHING:

P = pattern string : ABRA m

T = text string = ABRACADABRA n

If P occurs in T ...

BASELINE: for $i = 1 \dots n - m + 1$:
compare P with $T[i..i+m-1]$

$O(n \cdot m)$ time

KR - algorithm

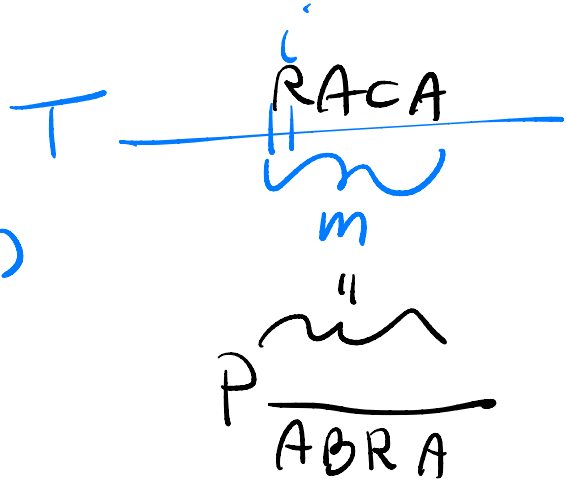
$p = F(P)$

$t = F(T[1..m])$

for $i = 1 \dots n - m + 1$:

compare p with t

update t in $O(1)$ time so that $t = F(T[i+1..i+m])$



$$X_i = \begin{cases} 1 & \text{error } p=t \text{ but } P \neq T[i..i+m-1] \\ 0 & \text{o.w.} \end{cases}$$

$$\Pr[X_i = 1] = \frac{m}{\tau / \ln \tau}$$

$$X = \sum_{i=1}^{n-m+1} X_i > 0 \text{ iff there is an error at some point}$$

$$\Pr(\text{error KR-alg}) \leq n \cdot \frac{m}{\tau / \ln \tau} \leq \frac{1}{nc}$$

$$1) m \leq n \Rightarrow mn \leq n^2$$

$$2) \tau \sim n^{c+2} \ln n$$

MONTESCARLO ALGORITHM

- running is guaranteed to be $O(n+m)$ time
- 1-Side error

LAS VEGAS ALGORITHM

- running time is expected
- no error

KR - algorithm: LAS VEGAS

if $(p=t)$ then if $P = T[i..i+m-1]$ then RETURN Ok

Running time is $O(n+m + \boxed{X} \cdot m)$

Expected running time is $O(n+m + E[X] \cdot m)$

$$\underbrace{\leq \frac{n^2}{\tilde{c} \rho_n \tilde{c}} \cdot m}_{O(n)}$$

10

